

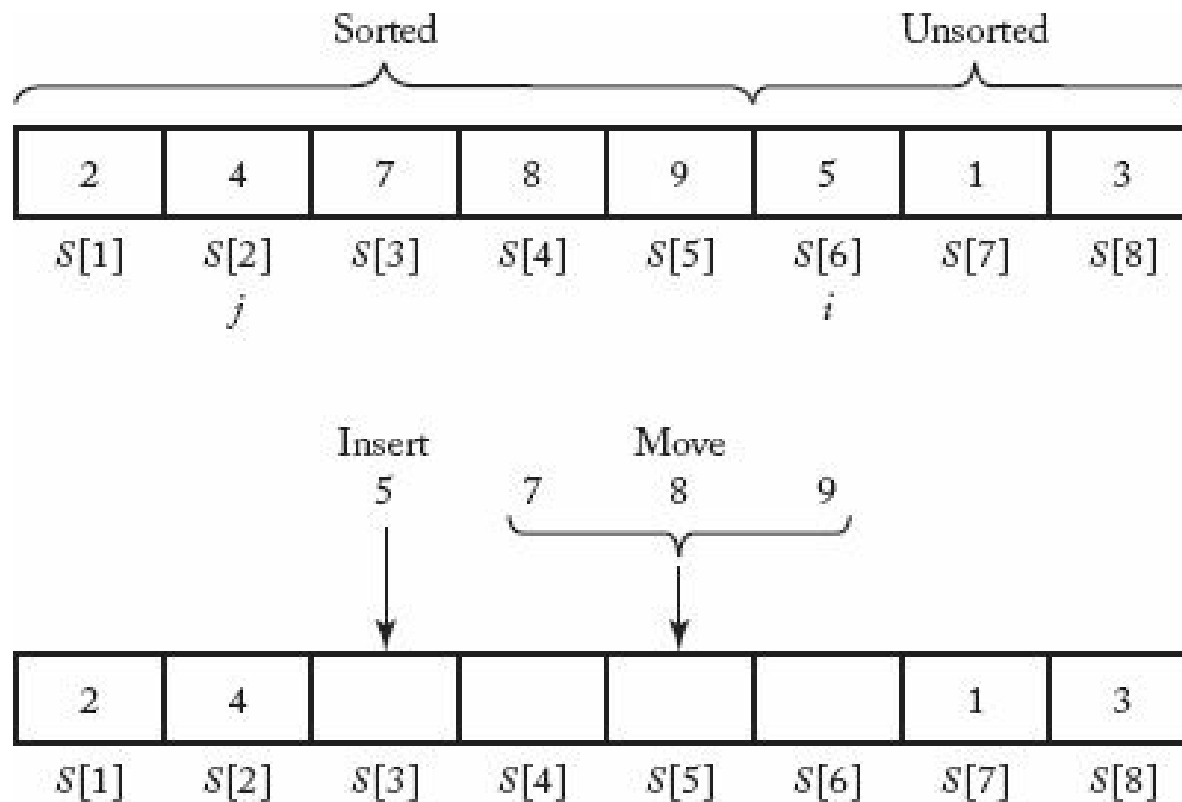
알고리즘

Chap. 정렬 문제 (Sorting Problem)

■ 삽입정렬 (Insertion Sort)

■ 삽입정렬

- 정렬된 배열에 데이터를 삽입하여 정렬하는 알고리즘
- Example ($i=6, j=2$)



■ 삽입정렬 (Insertion Sort)

- Algo 7.1 Insertion Sort
 - 문제: 비내림차순으로 n 개의 키를 정렬
 - 입력: 양의 정수 n , 키의 배열 $S[1..n]$
 - 출력: 키가 비내림차순으로 정렬된 배열 S

```
void insertionsort (int n, keytype S[])
{
    index i, j;
    keytype x;
    for (i = 2; i <= n; i++){
        x = S[i];
        j = i - 1;
        while (j > 0 && S[j] > x){
            S[j + 1] = S[j];
            j--;
        }
        S[j + 1] = x;
    }
}
```

■ 삽입정렬 (Insertion Sort)

- Algo 7.1(Insertion Sort)의 최악 시간 복잡도(Worst-case time complexity) 분석
 - 단위 연산: $s[j]$ 와 x 의 비교 연산
 - 입력 크기: 배열 s 의 원소 개수, n

i 에 대해서 비교는 $(i-1)$ 번 실행되고 $i=2, \dots, n$ 이므로, 총 최대 비교 횟수는

$$\sum_{i=2}^n (i-1) = \frac{1}{2}n(n-1)$$

즉, 배열 s 가 내림차순으로 정렬되어 있는 경우에 최대 비교 횟수 발생

$$W(n) = \frac{1}{2}n(n-1) \in \theta(n^2)$$

■ 선택정렬 (Selection Sort)

■ 교환정렬 (Exchange Sort)

- Algo 1.3 Exchange Sort

```
void exchange (int n, keytype S[]) {
```

```
    index i, j ;
```

```
    for (i = 1 ; i <= n-1 ; i++)
```

```
        for (j = i+1 ; j <= n ; j++)
```

```
            if ( S[j] < S[i] )
```

```
                exchange S[j] and S[i] ;
```

```
}
```

3	8	2	4	1	7	6	5
---	---	---	---	---	---	---	---

3	8	2	4	1	7	6	5
---	---	---	---	---	---	---	---

2	8	3	4	1	7	6	5
---	---	---	---	---	---	---	---

2	8	3	4	1	7	6	5
---	---	---	---	---	---	---	---

1	8	3	4	2	7	6	5
---	---	---	---	---	---	---	---

1	8	3	4	2	7	6	5
---	---	---	---	---	---	---	---

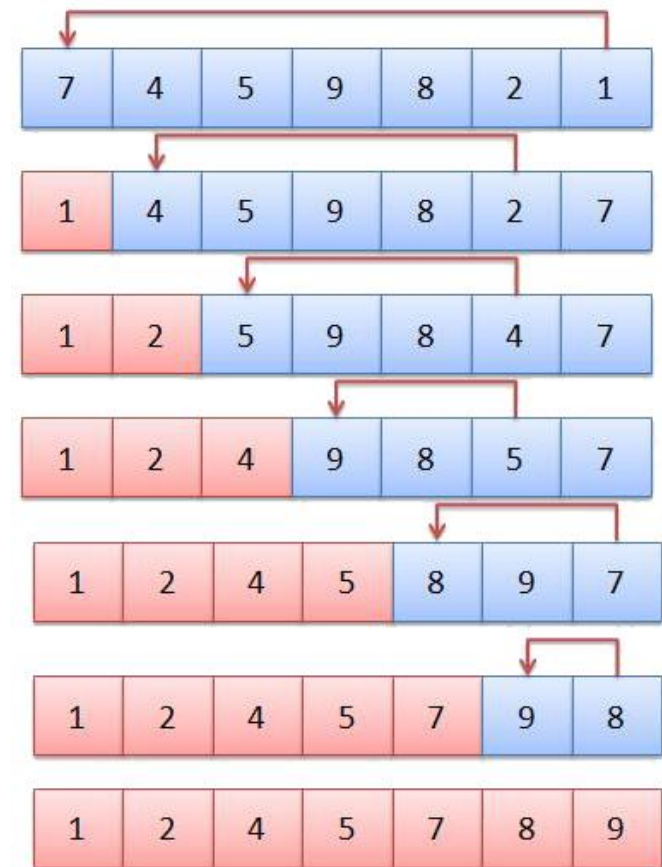
1	8	3	4	2	7	6	5
---	---	---	---	---	---	---	---

1	8	3	4	2	7	6	5
---	---	---	---	---	---	---	---

■ 선택정렬 (Selection Sort)

- Algo 7.2 Selection Sort
 - 문제: 비내림차순으로 n 개의 키를 정렬
 - 입력: 양의 정수 n , 키의 배열 $S[1..n]$
 - 출력: 키가 비내림차순으로 정렬된 배열 S

```
void selectionsort (int n, keytype S[])
{
    index i, j, smallest;
    for (i = 1; i <= n - 1; i++){
        smallest = i;
        for (j = i + 1; j <= n; j++)
            if (S[j] < S[smallest])
                smallest = j;
        exchange S[i] and S[smallest];
    }
}
```



■ 비교

Algorithm	Comparisons of Keys	Assignments of Records	Extra Space Usage
Exchange Sort	$T(n) = \frac{n^2}{2}$	$W(n) = \frac{3n^2}{2}$ $A(n) = \frac{3n^2}{4}$	In-place
Insertion Sort	$W(n) = \frac{n^2}{2}$ $A(n) = \frac{n^2}{4}$	$W(n) = \frac{n^2}{2}$ $A(n) = \frac{n^2}{4}$	In-place
Selection Sort	$T(n) = \frac{n^2}{2}$	$T(n) = 3n$	In-place

- 비교 횟수
 - 삽입정렬이 효율적 (교환정렬과 선택정렬은 동일함)
- 레코드 저장 횟수
 - 선택정렬 (1차) < 삽입정렬 (2차) < 교환정렬 (2차)

■ 비교

- 비교 횟수
 - 삽입정렬이 효율적 (교환정렬과 선택정렬은 동일함)
- 레코드 저장 횟수
 - 선택정렬 (1차) < 삽입정렬 (2차) < 교환정렬 (2차)
- 선택정렬 (Selection Sort)
 - $S[i] > S[j]$ 경우, j 의 index 저장
 - 첫번째 for-i 루프 실행 후, 가장 작은 값을 $S[i]$ 와 교환
 - 전체 교환 횟수는 $n-1$ 번 (교환 횟수 1번은 레코드 저장 횟수 3번)
 - 레코드 저장 횟수는 $T(n) = 3(n-1)$
- 교환정렬 (Exchange Sort)
 - $S[i] > S[j]$ 경우, $S[i]$ 와 $S[j]$ 를 교환
 - 레코드 저장 횟수는 $3n^2/4$
 - 입력 배열 s 가 이미 정렬된 경우에는 저장 작업을 수행하지 않음